```
# Load libraries

suppressWarnings(
  library(caret)
  library(dplyr)
  library(pROC)
  library(xgboost)
  library(randomForest)

  # Set seed
  set.seed(123)

  predictor_cols <- c(
    "avg_steps", "avg_calories_out", "avg_bmr", "avg_sedentary",
    "avg_very_active", "avg_lightly_active", "avg_fairly_active",
    "avg_floors", "avg_elevation",
    "avg_hr_min", "avg_hr_max", "total_minutes_in_zone",
"avg_calories_hr",
    "avg_sleep_minutes", "avg_time_in_bed",
    "avg_light_sleep", "avg_rem_sleep", "avg_wake_minutes",
    "systolic_bp", "diastolic_bp", "BMI"
  )

  label_cols <- c("osteoporosis", "dementia", "heart_failure",
"parkinsonism")

  # Ensure all columns exist
  stopifnot(all(c(predictor_cols, label_cols) %in%
names(joined_data)))

  # Filter complete cases
  df_clean <- joined_data %>%
    filter(if_all(all_of(predictor_cols), ~ !is.na(.)))

  # Loop through conditions



  for (label in label_cols) {

    cat("\n========== Modeling for", label, "==========\n")

    # Prepare data
    data_model <- df_clean %>%
      select(all_of(c(label, predictor_cols)))

    data_model[[label]] <- factor(data_model[[label]], levels = c(0,
1), labels = c("No", "Yes"))

    # Manual stratified train/test split to guarantee "Yes" in test
```

```
set
    yes_idx <- which(data_model[[label]] == "Yes")
    no_idx  <- which(data_model[[label]] == "No")

    test_yes <- sample(yes_idx, size = min(90, length(yes_idx)))
    test_no  <- sample(no_idx, size = min(400, length(no_idx)))

    test_index <- c(test_yes, test_no)
    train_index <- setdiff(1:nrow(data_model), test_index)

    train_data <- data_model[train_index, ]
    test_data  <- data_model[test_index, ]

    cat("Train set balance:\n")
    print(table(train_data[[label]]))

    cat("Test set balance:\n")
    print(table(test_data[[label]]))

    # Set up CV with upsampling
    ctrl <- trainControl(
      method = "cv",
      number = 5,
      summaryFunction = twoClassSummary,
      classProbs = TRUE,
      savePredictions = "final",
      sampling = "up"
    )

    # Train models
    logit_model <- train(
      reformulate(predictor_cols, response = label),
      data = train_data,
      method = "glm",
      family = "binomial",
      trControl = ctrl,
      metric = "ROC"
    )

    rf_model <- train(
      reformulate(predictor_cols, response = label),
      data = train_data,
      method = "rf",
      trControl = ctrl,
      metric = "ROC"
    )

    xgb_model <- train(
      reformulate(predictor_cols, response = label),
      data = train_data,
```

```
      method = "xgbTree",
      trControl = ctrl,
      metric = "ROC",
      tuneLength = 5
    )

    # Evaluate
    models <- list(Logit = logit_model, RF = rf_model, XGB =
xgb_model)

    for (model_name in names(models)) {
      model <- models[[model_name]]
      probs <- predict(model, newdata = test_data, type = "prob")[,
"Yes"]
      preds <- predict(model, newdata = test_data)

      roc_obj <- roc(test_data[[label]], probs)
      auc_val <- auc(roc_obj)

      cm <- confusionMatrix(preds, test_data[[label]], positive =
"Yes")
      acc <- cm$overall["Accuracy"]

      precision <- cm$byClass["Precision"]
      recall    <- cm$byClass["Recall"]
      f1 <- if (is.na(precision) || is.na(recall) || precision +
recall == 0) {
        NA
      } else {
        2 * (precision * recall) / (precision + recall)
      }

      cat(sprintf("\n--- %s ---\n", model_name))
      cat(sprintf("Accuracy: %.3f | F1: %s | AUC: %.3f\n",
                  acc,
                  ifelse(is.na(f1), "Undefined (No Positives)",
sprintf("%.3f", f1)),
                  auc_val))
    }

    # Feature importance
    cat("\nTop Features:\n")
    cat("XGBoost:\n")
    print(varImp(xgb_model)$importance %>% arrange(desc(Overall)) %>%
head(5))

    cat("\nRandom Forest:\n")
    print(varImp(rf_model)$importance %>% arrange(desc(Overall)) %>%
head(5))
```

```
    cat("\n==============================\n")
  }
)
```